

UZF 421 E - Attitude Determination and Control

Homework Assignment 3

Ahmet Fevzi Akargöl
Student ID: 110200170

May 4, 2025

Task 1: Magnetometer Sensor Research

Search for three different magnetometer sensors used in spacecraft missions. Fill the Table 1 for the related sensors' characteristics. At least 5 of the items need to be filled for each sensor.

Table 1: Characteristics of the magnetometer samples.

	Sensor A (GomSpace M315)	Sensor B (AAC Clyde MAG-3)	Sensor C (AAC Clyde MM200)
Website of the Datasheet	PDF Link	PDF Link	PDF Link
Sensor Type	3-Axis Magnetometer	Fluxgate	High Precision Magnetometer
Size in m	$0.023 \times 0.020 \times 0.008$	$\sim$ $0.083 \times 0.035 \times 0.032$	$0.020 \times 0.020 \times 0.011$ (Min)
Mass in kg	0.008	0.1	0.010 (Min)
Operating Temperature in °C	-40 to +85	-55 to +85	-
Range in nT	$\pm 800,000$	$\pm 65,000$ (std)	$\pm 800,000$
Standard Deviation in nT	< 1 (1-sigma)	-	-

Note: Values are approximate or based on available data. Standard deviation might be specified differently (e.g., noise density) in datasheets. Please refer to datasheets for exact specifications.

Task 2 & 3: Simulation and Raw Measurement Plots

Three-axis magnetometer simulation using the equation $B_b = CB_N + b + v$. Where B_N is the assumed magnetic field, C is the DCM from HW1 ($n=0$), b is the bias vector ($n=0$), and v is the noise vector ($\sigma = 100$ nT).

The magnetic field measurements (B_b) were computed for $N=6000$ samples for the following 3 cases:

- $B_b^{\text{Case 1}} = CB_N + b + v$
- $B_b^{\text{Case 2}} = CB_N + v$
- $B_b^{\text{Case 3}} = CB_N$

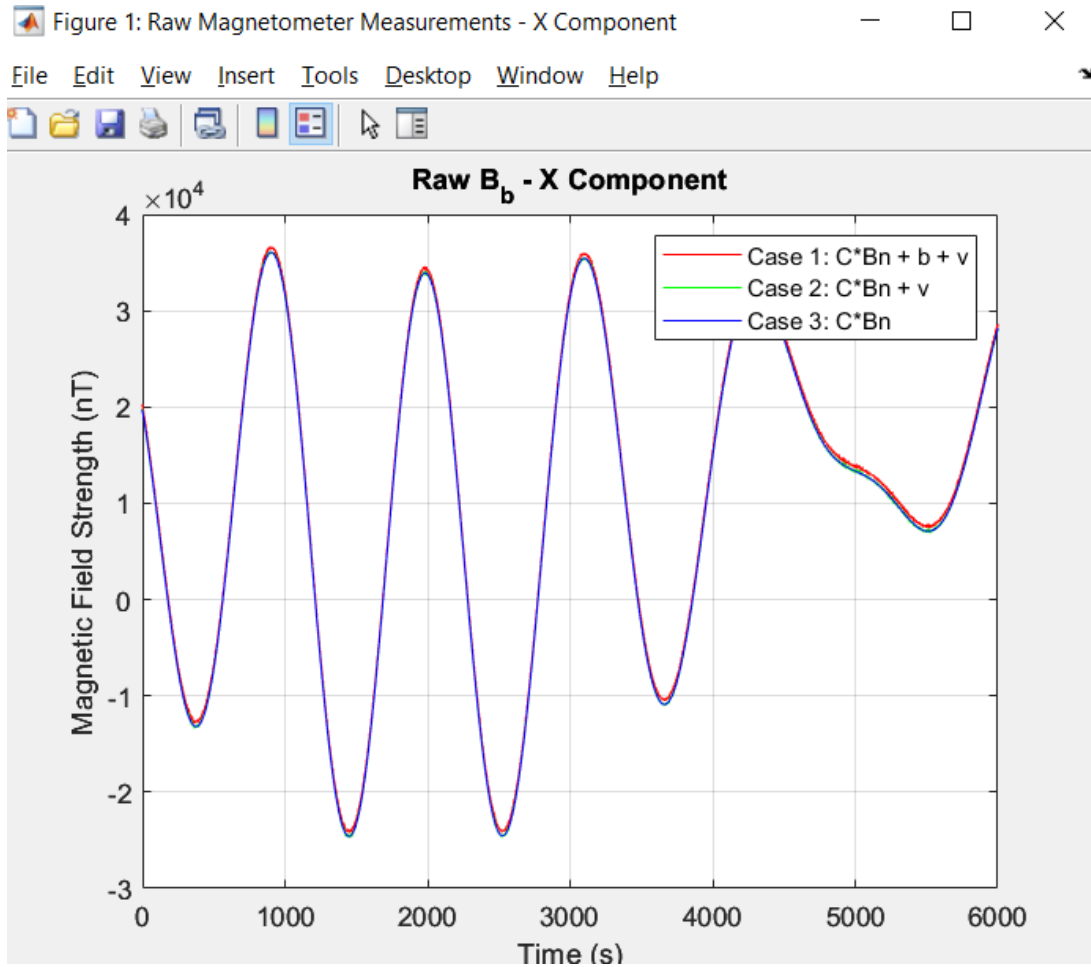


Figure 1: Raw Magnetic Field Measurements (B_b) for X-Component (Cases 1, 2, 3).

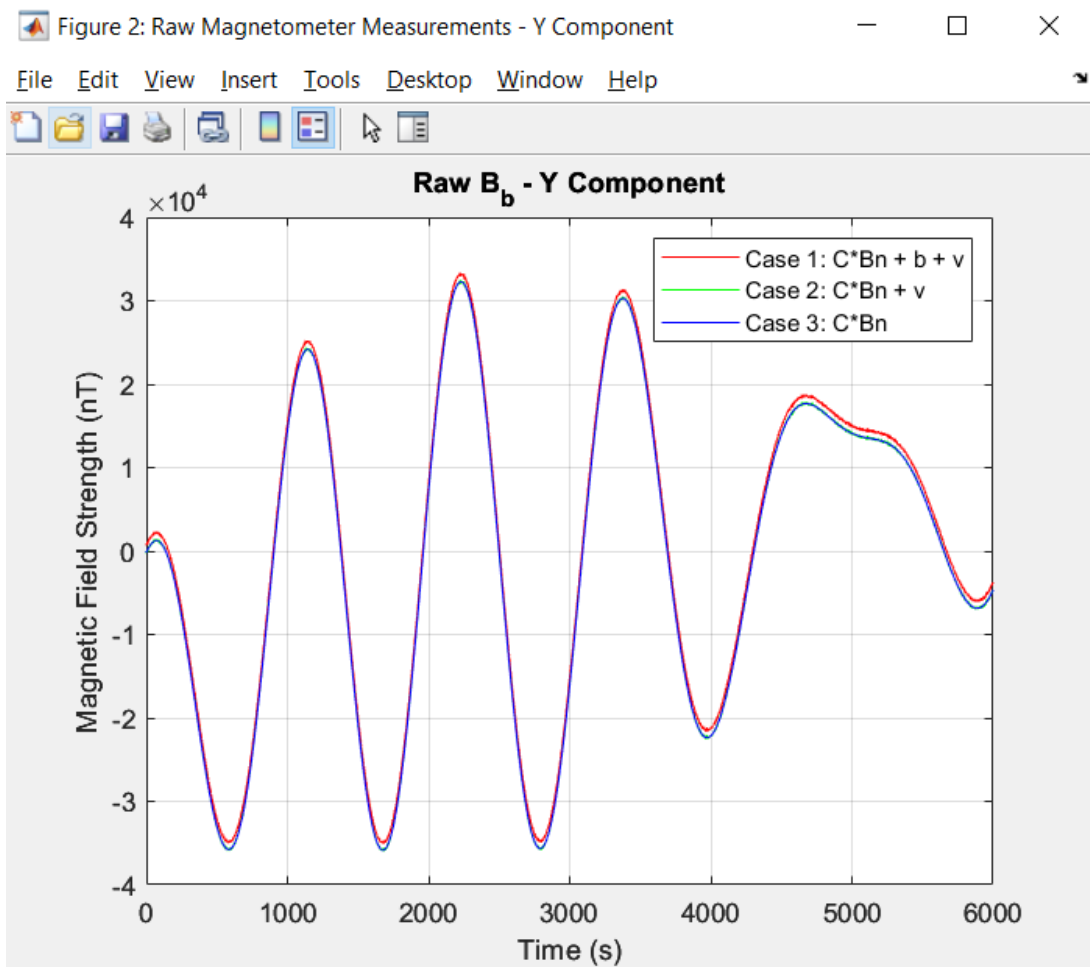


Figure 2: Raw Magnetic Field Measurements (B_b) for Y-Component (Cases 1, 2, 3).

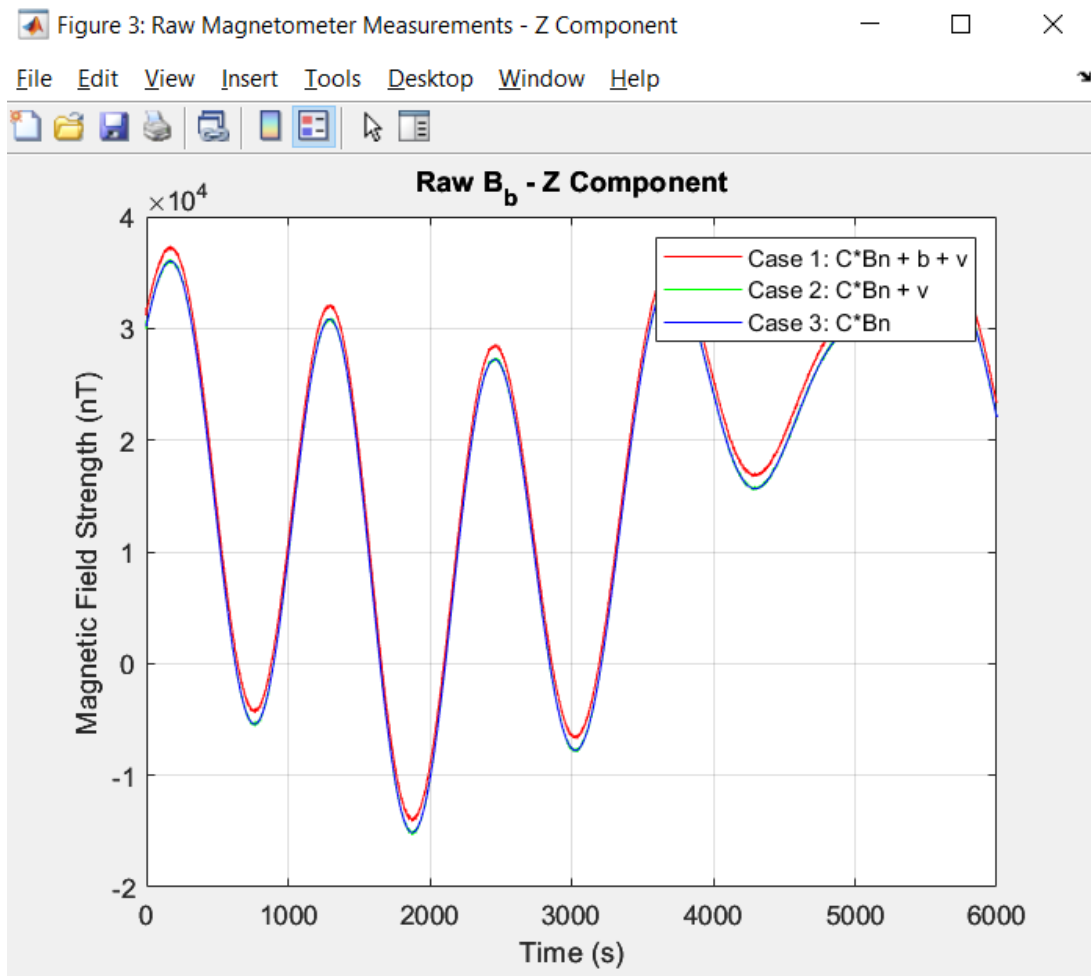


Figure 3: Raw Magnetic Field Measurements (B_b) for Z-Component (Cases 1, 2, 3).

Task 4: Normalized Measurement Plots

The computed B_b vectors for all three cases were divided by their respective norms ($\|B_b\|$). The components of these normalized vectors are plotted below.

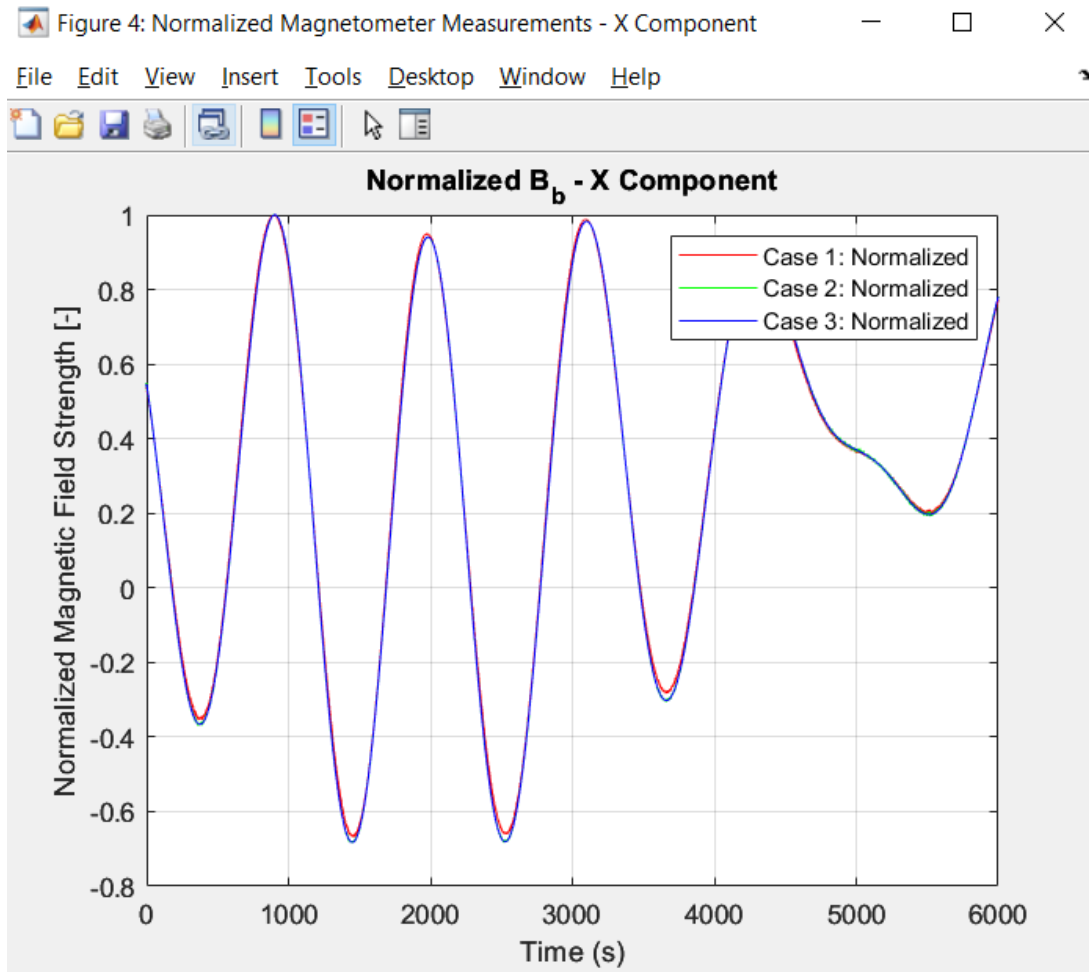


Figure 4: Normalized Magnetic Field Measurements ($B_b/\|B_b\|$) for X-Component (Cases 1, 2, 3).

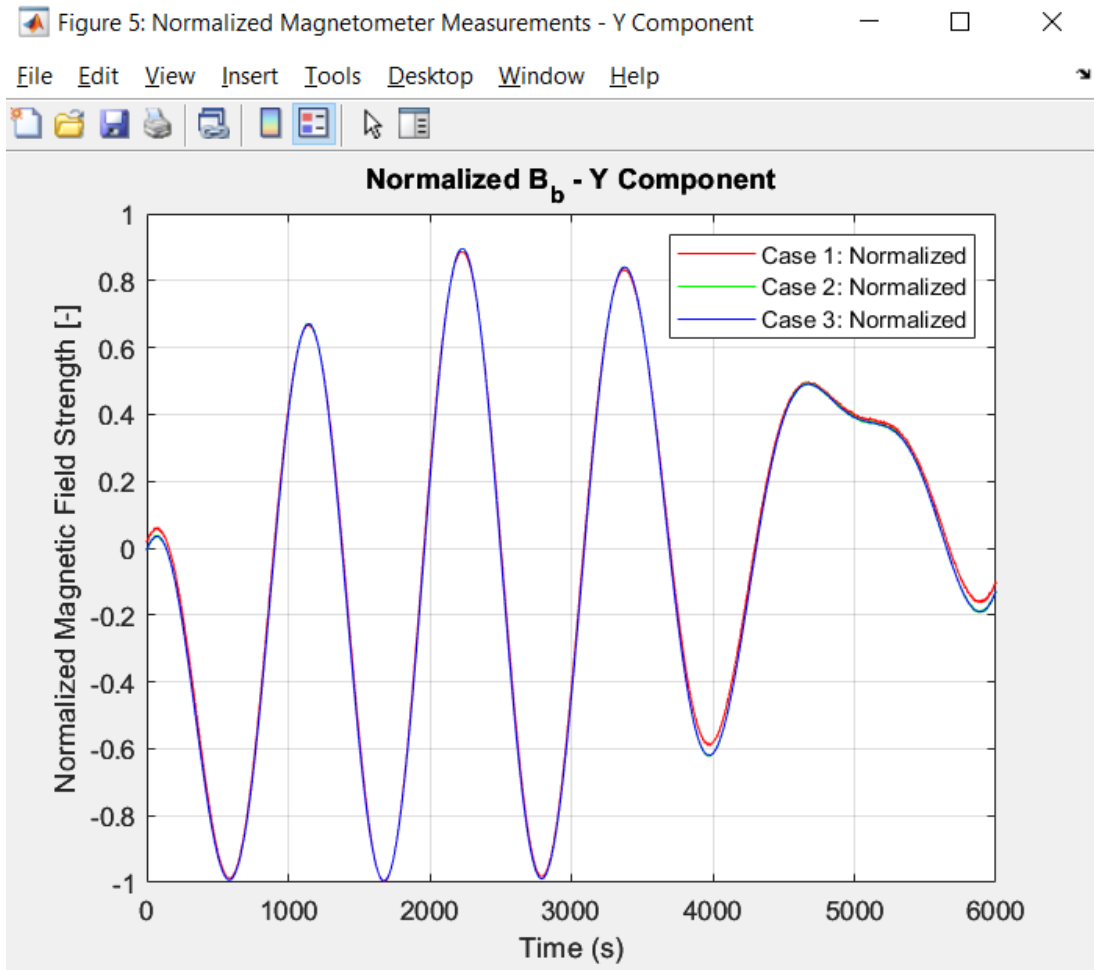


Figure 5: Normalized Magnetic Field Measurements ($B_b/||B_b||$) for Y-Component (Cases 1, 2, 3).

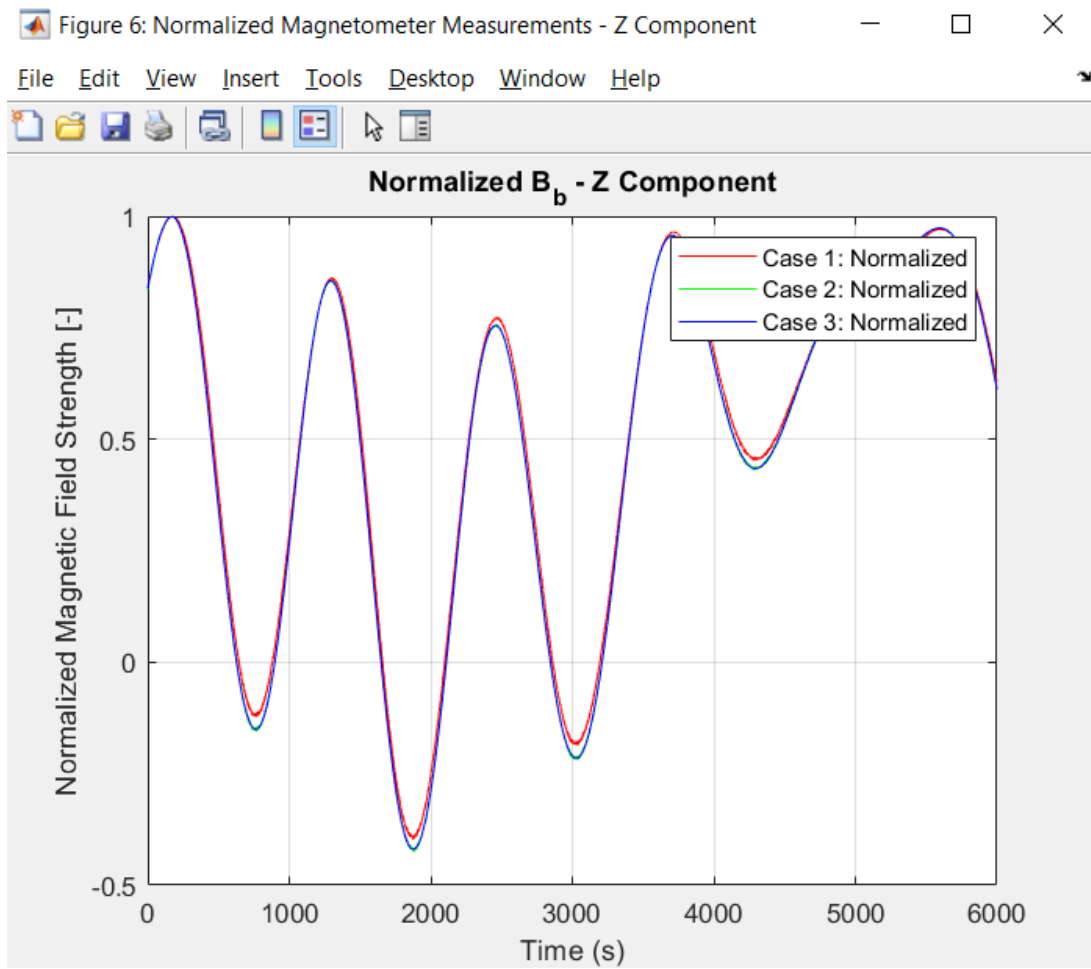


Figure 6: Normalized Magnetic Field Measurements ($B_b/||B_b||$) for Z-Component (Cases 1, 2, 3).

Task 5: Explanation for Bias and Noise

What is the reason of adding bias or noise terms on top of the transformed magnetic field vector?

We add bias (b) and noise (v) terms to the ideal transformed magnetic field vector (CB_N) to make our simulation model, $B_b = CB_N + b + v$, behave more like a real-world magnetometer sensor. Here's why:

- **Real Sensors Aren't Perfect:** Physical sensors like magnetometers always have some imperfections and are affected by their environment. They don't measure the true magnetic field exactly as it is after being rotated into the sensor's frame (CB_N).
- **Bias (b):** This term represents systematic errors that consistently push the measurement off from the true value. For a magnetometer, this could be caused by:
 - The sensor itself having a slight permanent magnetization or calibration error.
 - Magnetic fields generated by other parts of the satellite (like currents in wires, magnets in actuators, etc.) interfering with the measurement.

This error is usually constant or changes very slowly over time. Adding ' b ' accounts for this persistent offset.

- **Noise (v):** This term represents the random, unpredictable fluctuations that occur in any electronic measurement. Think of it like static or fuzziness on top of the signal. It can be caused by:
 - Thermal noise within the sensor's electronics.
 - Quantization errors (converting the analog signal to digital).
 - Rapid, small changes in the surrounding magnetic environment.

Adding ' v ' simulates this inherent randomness and uncertainty in the sensor reading at any given moment.

In summary, including bias and noise makes our simulation much more realistic. It acknowledges that real sensor data isn't perfectly clean or accurate. This is important because algorithms that use these measurements (like attitude determination algorithms) need to be able to handle these imperfections found in actual flight data. By simulating these errors, we can better test and design robust systems.

A Appendix: MATLAB Code

```
clc; clear; close all;

%% Constants definitions
n = 0; % Last digit of student number
N = 6000; % Number of samples
delta_t = 1; % Sampling time (seconds)
time = (0:N)*delta_t; % Time vector from 0 to N seconds

%
-----

% SECTION FROM HOMEWORK 1 (for n=0) - Calculating DCM (C Matrix)
%
-----

% Initial attitude angles (rad) for n=0
psi_0 = -0.005 - 0.002*n; % Yaw
theta_0 = 0.01 - 0.002*n; % Pitch
phi_0 = -0.01 - 0.002*n; % Roll

% Initial angular velocities (rad/s) for n=0
w1_0 = -0.002 - 0.0001*n;
w2_0 = 0.003 - 0.0001*n;
w3_0 = -0.004 - 0.0001*n;

% Mass moments of inertia (kg.m^2)
I1 = 2.1e-3;
I2 = 2.0e-3;
I3 = 1.9e-3;

% Disturbance torques (N.m)
L1 = 3.6e-10;
L2 = 3.6e-10;
L3 = 3.6e-10;

% --- Propagate Angular Velocities (Euler Method) ---
w = [w1_0, w2_0, w3_0];
w_history = zeros(N+1, 3);
w_history(1, :) = w;

for i = 1:N
    dw1 = (-(I3 - I2)*w(2)*w(3) + L1) / I1;
    dw2 = (-(I1 - I3)*w(3)*w(1) + L2) / I2;
    dw3 = (-(I2 - I1)*w(1)*w(2) + L3) / I3;

    % Euler integration step for omega
    w = w + [dw1, dw2, dw3] * delta_t;
    w_history(i+1, :) = w;
end

% --- Propagate Euler Angles (Euler Method - Based on HW1 Eq. 2) ---
% Using the 3-2-1 sequence
euler_history = zeros(N+1, 3); % Stores [psi, theta, phi] in columns
euler_history(1, :) = [psi_0, theta_0, phi_0];
```

```

for i = 1:N
    psi_k    = euler_history(i, 1);
    theta_k  = euler_history(i, 2);
    phi_k    = euler_history(i, 3);
    w_k      = w_history(i, :)'; % Use omega at the beginning of the
        interval

    % Kinematic Matrix (3-2-1 sequence)
    % Check for singularity near cos(theta) = 0
    if abs(cos(theta_k)) < 1e-8
        warning('Near singularity in Euler angles at step %d', i);
        dEuler_dt = zeros(3,1); % Assign zero derivatives
    else
        Kinematic_Matrix = (1/cos(theta_k)) * [ 0, sin(phi_k),
                                                cos(phi_k);
                                                0, cos(phi_k)*cos(
                                                    theta_k), -sin(phi_k)
                                                    *cos(theta_k);
        cos(theta_k), sin(phi_k)
        *sin(theta_k), cos(
        phi_k)*sin(theta_k)
        ];

        dEuler_dt = Kinematic_Matrix * w_k;
    end

    % Euler integration step for angles [psi, theta, phi]
    euler_history(i+1, :) = euler_history(i, :) + dEuler_dt * delta_t;
end

% --- Calculate DCM (C Matrix - Based on HW1 Section C / B.a results)
% ---
% Compute C based on the 3-2-1 Euler angles [psi, theta, phi] calculated
    above.
% DCM for 3-2-1 (Z-Y-X) rotation sequence is C = R_x(phi) * R_y(theta) *
    R_z(psi)
C_dcm = zeros(3, 3, N + 1);
for i = 1:(N + 1)
    psi    = euler_history(i, 1);
    theta  = euler_history(i, 2);
    phi    = euler_history(i, 3);

    cos_psi = cos(psi); sin_psi = sin(psi);
    cos_theta = cos(theta); sin_theta = sin(theta);
    cos_phi = cos(phi); sin_phi = sin(phi);

    % Rotation matrices
    Rz = [ cos_psi, sin_psi, 0; -sin_psi, cos_psi, 0; 0, 0, 1 ];
    Ry = [ cos_theta, 0, -sin_theta; 0, 1, 0; sin_theta, 0, cos_theta ];
    Rx = [ 1, 0, 0; 0, cos_phi, sin_phi; 0, -sin_phi, cos_phi ];

    % DCM from Inertial (N) to Body (B) is C_BN = Rx * Ry * Rz
    C_dcm(:, :, i) = Rx * Ry * Rz;
end

%
-----

% HOMEWORK 3 CALCULATIONS

```

```

% -----

% --- Define Constants for HW3 ---
b = [500; 900; 1200] + 100*n; % Bias vector for n=0 (nT)
sigma = 100; % Noise standard deviation (nT)

% --- Assume Bn (Magnetic Field in Inertial Frame) ---
% NOTE: This is an assumed Bn vector as HW2 results were not available.
Bn = zeros(3, N+1);
Orbit_Period_Assumption = 6000; % Assumed orbital period in seconds for
    variation
for j = 1:(N+1)
    t_current = time(j);
    Bn(1, j) = 20000 * cos(2 * pi * t_current / Orbit_Period_Assumption)
        ;
    Bn(2, j) = 20000 * sin(2 * pi * t_current / Orbit_Period_Assumption)
        ;
    Bn(3, j) = 30000; % Constant Z component assumption
end
% Bn is assumed to be in nT units.

% --- Generate Noise Vector (v) ---
v_noise = sigma * randn(3, N+1); % Zero-mean Gaussian white noise (nT)

% --- Calculate Magnetometer Measurements (Bb) for 3 Cases ---
Bb_case1 = zeros(3, N+1); % Bb = C*Bn + b + v
Bb_case2 = zeros(3, N+1); % Bb = C*Bn + v
Bb_case3 = zeros(3, N+1); % Bb = C*Bn

for j = 1:(N+1)
    % Rotation matrix from N to B
    C_current = C_dcm(:, :, j);
    Bn_current = Bn(:, j);

    Bb_case3(:, j) = C_current * Bn_current;
    Bb_case2(:, j) = Bb_case3(:, j) + v_noise(:, j);
    Bb_case1(:, j) = Bb_case3(:, j) + b + v_noise(:, j); % Bias 'b' added
        here
end

% --- Plot Raw Magnetic Field Measurements (Task 3) ---
% Figure generation commands omitted here for brevity in Appendix,
% but they exist in the execution part above. Add them back if needed in
    report.
% figure('Name', 'Raw Magnetometer Measurements - X Component'); ...

% --- Normalize Bb Vectors (Task 4) ---
Bb_norm_case1 = zeros(3, N+1);
Bb_norm_case2 = zeros(3, N+1);
Bb_norm_case3 = zeros(3, N+1);

for j = 1:(N+1)
    norm1 = norm(Bb_case1(:, j)); norm2 = norm(Bb_case2(:, j)); norm3 =
        norm(Bb_case3(:, j));
    if norm1 > 1e-10, Bb_norm_case1(:, j) = Bb_case1(:, j) / norm1; end
    if norm2 > 1e-10, Bb_norm_case2(:, j) = Bb_case2(:, j) / norm2; end

```

```

    if norm3 > 1e-10, Bb_norm_case3(:,j) = Bb_case3(:,j) / norm3; end
end

% --- Plot Normalized Magnetic Field Measurements (Task 4) ---
% Figure generation commands omitted here for brevity in Appendix.
% figure('Name', 'Normalized Magnetometer Measurements - X Component');
...

%
-----

% END OF MATLAB CODE
%
-----

```

Listing 1: MATLAB Code for Homework 3 Simulation ($n=0$, Assumed B_N)